Algebraic Smooth Occluding Contours

Ryan Capouellez
New York University
USA
rjc8237@nyu.edu

Jiacheng Dai
New York University
USA
jd4705@nyu.edu

Aaron Hertzmann
Adobe
USA
hertzman@dgp.toronto.edu

Denis Zorin
New York University
USA
dzorin@cs.nyu.edu

Figure 1: Our method takes a triangle mesh, and renders smooth occluding contours for a given camera viewpoint. For each view, the algorithm produces a piecewise-quadratic surface, for which we show the occluding contours can be computed algebraically under orthographic projection. We use a projective transformation of the input mesh to handle perspective projection. By precomputing a conformal parameterization and a mapping from the viewpoint to the quadratic coefficients, contours and visibility can be computed very efficiently, in contrast to previous methods that used expensive heuristics or computations to produce smooth contours with accurate visibility. (Public domain Spot model by Keenan Crane.)

ABSTRACT

Computing occluding contours is a key step in 3D non-photorealistic rendering, but producing smooth contours with consistent visibility has been a notoriously-challenging open problem. This paper describes the first general-purpose smooth surface construction for which the occluding contours can be computed in closed form. Given an input mesh and camera viewpoint, we show how to approximate the mesh with a G^1 piecewise-quadratic surface, for which the occluding contours are piecewise-rational curves in image-space. We show that this method produces smooth contours with consistent visibility much more efficiently than the state-of-the-art.

CCS CONCEPTS

• **Computing methodologies → Non-photorealistic rendering**; **Visibility**; **Parametric curve and surface models**.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
SIGGRAPH '23 Conference Proceedings, August 6–10, 2023, Los Angeles, CA, USA
© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-0159-7/23/08…$15.00
https://doi.org/10.1145/3588432.3591547

KEYWORDS

contours, non-photorealistic rendering, piecewise-quadratic surface, visibility

ACM Reference Format:
Ryan Capouellez, Jiacheng Dai, Aaron Hertzmann, and Denis Zorin. 2023. Algebraic Smooth Occluding Contours. In *Special Interest Group on Computer Graphics and Interactive Techniques Conference Conference Proceedings (SIGGRAPH '23 Conference Proceedings), August 6–10, 2023, Los Angeles, CA, USA.* ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/3588432.3591547

1 INTRODUCTION

Computing occluding contour lines of 3D objects is a common step in 3D non-photorealistic rendering algorithms, whether for architectural drawings, cartoon stylization, or pen-and-ink illustration. Conceptually, the problem is deceptively simple: find the points of the surface where the dot product $\mathbf{n} \cdot \tau$ of the view direction τ and the normal vector $\mathbf{n}$ changes sign, and determine which of these points are visible [Bénard and Hertzmann 2019].

For triangle meshes, the exact occluding contours can easily be defined: they are a subset of the edges of the original mesh. However, when the mesh represents a smooth object, these contours usually have many spurious singularities, and do not produce the clean contour topology of a smooth surface (Figure 2). This makes them unsuitable for curve stylization, and noisy during animation, e.g., [Bénard et al. 2014]. This mismatch between mesh contours

and smooth surface contours can be attributed to the fact that the normal $\mathbf{n}$ is discontinuous on the mesh, and sign-change sets of discontinuous functions fundamentally differ in structure from the zero sets of smooth functions.

Unfortunately, robustly computing the occluding contours for smooth surfaces is difficult. For common representations, the occluding contours are projective-transformed piecewise higher-order algebraic implicit curves. Existing methods approximate these curves with polylines, but visibility for these polylines is unreliable, as they are not the contours of the smooth surface [Bénard et al. 2014; Eisemann et al. 2008]. Recent methods construct a new triangle mesh with the extracted polylines as contours [Bénard et al. 2014; Liu et al. 2023], but require very costly heuristic search. These difficulties raise the question: is there a practical smooth representation for which visible occluding contours can be computed exactly?

In this paper, we describe a method for computing occluding contours in *closed form*. For a given input mesh and camera viewpoint, we show how to approximate the mesh with a C^1 (excluding a small set of points) surface so that the occluding contours are piecewise-rational curves in image space. This algebraic representation allows for reliable visibility by solving low-order polynomial equations, and for direct, efficient computations without heuristics or search.

Our approach operates as follows. We first apply a projective transformation to the input mesh, reducing the problem to orthographic occluding contours. We use a Powell-Sabin construction to produce a C^1 *quadratic patch* surface, which, we observe, is the only algebraic representation with rational contours. To support perspective views, our surface construction is *view-dependent* but can be evaluated very efficiently with a precomputed matrix factorization. All possible piecewise rational quadratic contour lines in the parametric domain are easily enumerated, and correspond to piecewise rational quartic curves in the image domain. Visibility for the contours is also resolved precisely by a straightforward computation (up to numerical errors in solving low-order polynomial equations).

In proposing the first algebraic smooth occluding contour procedure, our main contribution is a careful integration and adaptation of a number of recent and classical techniques in a highly-constrained setting. This includes recent methods for robust surface parameteriztion, a view-dependent almost-everywhere C^1 surface approximation, supporting efficient per-view updates with precomputation, quantitative invisibility for piecewise quadratic surfaces, and efficient cusp computation.

In comparison to state-of-the-art methods for accurate contours [Bénard et al. 2014; Liu et al. 2023], our method does not involve expensive search and unpredictable refinement heuristics; we find order-of-magnitude faster performance on larger meshes, while avoiding the visibility errors that older methods are prone to.

2 RELATED WORK

It has long been known that the occluding contours for a smooth surface cannot be computed analytically, except for simple primitives like spheres and quadrics [Cipolla and Giblin 2000]. Hence, all previous methods employ numerical approximations of the smooth contour; see [Bénard and Hertzmann 2019] for a survey. Past methods either often produce artifacts, or else require expensive computations to achieve topologically-accurate curves.

(a) (b)

Figure 2: While contour lines on piecewise-linear surfaces can be computed robustly and efficiently, these consist of a noisy set of mesh edges (b), not suitable for constructing, e.g., vector approximations of contours or stylized rendering; in comparison, smooth surface contours (a) typically yield a clean spatial curve. Figure courtesy of the authors of [Liu et al. 2023].

Raster methods, based on edge detection of an image buffer are the simplest way to approximate the contour, e.g., [Decaudin 1996; Saito and Takahashi 1990]; however, these methods do not produce a vector representation of the contours.

Representing a smooth surface as a triangle mesh yields accurate image-space contours but with erroneous topology (Figure 2). Heuristics may be used to smooth the topology for stylization [Eisemann et al. 2008; Isenberg et al. 2002; Kirsanov et al. 2003; Northrup and Markosian 2000], but these too may be very inaccurate and often produce artifacts.

A third approach is to directly compute a polyline contour approximation, e.g., by root-finding on the smooth surface representation [Elber and Cohen 1990; Stroila et al. 2008; Weiss 1966; Winkenbach and Salesin 1996]. However, visibility of these sampled contours are inconsistent with the smooth surface [Bénard et al. 2014]. Defining a piecewise-smooth contour function on a triangle mesh [Hertzmann and Zorin 2000] has the same problem. Bénard et al. [2014] generate a new triangle mesh from sampled contours that produces consistent visibility. However, this method has a high computational cost and is not guaranteed to find a valid mesh. Planar-maps can produce consistent visibility [Stroila et al. 2008; Winkenbach and Salesin 1996], but may also include incorrect topology from polyline contour approximations.

Liu et al. [2023] recently explained why this problem has been so difficult: sampling smooth contours produces 2D polylines that cannot be the contours of any valid surface. Liu et al. do guarantee valid polylines, but their method uses expensive numerical sampling operations, involving costly iterative refinements and heuristics to find a consistent mesh.

In contrast to these works, we present a method using a C^1 approximation of an arbitrary input mesh for which we can compute *exact* (up to numerical precision) contours algebraically, with precise visibility, without any of the heuristics or expensive refinement procedures of previous methods.

The geometric modeling literature on constructing G^1/C^1 surfaces is vast, but due to constraints of our problem, only a few constructions are relevant in our context; [He et al. 2005; Powell and Sabin 1977] which we rely on, and [Dahmen 1989] being most closely related. We provide more details in Section 5.

Linear-normal (LN) surfaces [Jüttler 1998; Jüttler and Sampoli 2000] produce G^1 surfaces with normals depending linearly on the parametric coordinates, which, in turn, yields piecewise-linear contour lines in the parametric domain, but they are limited in the type of surfaces that these can represent, e.g., they have singularities at parabolic points.

3 OVERVIEW

Given an input mesh M and camera position $\mathbf{c}$, we seek a tangent plane-continuous surface representation that approximates the mesh and yields high-quality contours that are continuous and have explicit algebraic form, allowing exact and efficient computation. Our key observation is that the problem of finding this representation is extremely constrained—yet it does have a solution. We start with defining the problem more precisely.

We use $\mathbf{p}(u,v) : \Omega \subset \mathbb{R}^2 \rightarrow \mathbb{R}^3$ to denote a surface patch parameterized over a domain Ω in the plane. The parameterization is assumed to be at least C^1. The normal $\mathbf{n}(u,v) = \mathbf{p}_u \times \mathbf{p}_v$ is not necessarily unit length. The camera position is $\mathbf{c}$, and the image plane is P, with unit normal τ.

The *occluding contour* of the image of the surface patch $\mathbf{p}$ in P is the projection to the image plane of the set of points $\mathbf{p}(C)$, with the curve C in the parametric domain defined by

$$\mathbf{n}(u,v)\cdot(\mathbf{c}-\mathbf{p}(u,v)) = 0 \text{ (perspective)}, \ \mathbf{n}(u,v)\cdot\tau = 0 \text{ (orthographic)} \tag{1}$$

The curve $\mathbf{p}(C)$ is the *occluding contour generator*, and the apparent contour is the visible projection of that curve. For brevity, we refer to each of these 2D and 3D curves as "contours." These elements are illustrated in Figure 3. Existing methods for solving these equations employ numerical approximations, as no closed-form solution is known for these equations for general-purpose smooth surfaces.

Algebraic Contour Existence. Under what conditions does Equation 1 yield curves in closed-form? For algebraic functions, the following is known, e.g., [Ferrer et al. 2008]:

Proposition 1. *An irreducible algebraic curve $c(u,v) = 0$ admits a rational parameterization for an arbitrary choice of coefficients if and only if it is linear or quadratic.*

The linear case corresponds to using a triangle mesh, which has been heavily used in previous methods, with the problems discussed in Section 2. Juttler et al. [Jüttler 1998; Jüttler and Sampoli 2000] describe a higher-order algebraic surface with constraints that make C reducible into a linear factor and another factor independent of the view direction, but the construction is degenerate at parabolic points. Instead, we consider surfaces where $\mathbf{p}(u,v)$ is *quadratic*, which has not been explored in depth. When $\mathbf{p}(u,v)$

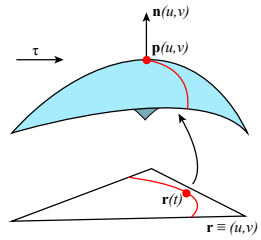

Figure 3: Left: The occluding contours of a smooth patch are the points $\mathbf{p}(u,v)$ where the normal and view direction are orthogonal ($\mathbf{n}(u,v)\cdot\tau = 0$). In this paper, we show that the contour for a quadratic patch can be described by an algebraic curve $\mathbf{r}(t)$ in the parameter domain, so the 3D contour is $\mathbf{p}(\mathbf{r}(t))$. Right: 3D perspective transformation converting perspective projection frustum to a cube. (Public domain Spot model by Keenan Crane.)

is quadratic, $\mathbf{n}(u,v)$ is also quadratic. Then, for orthographic projection, Equation 1 is quadratic, and thus the solution curves are rational functions. It is also clear from Equation 1 that contour continuity requires that the surface is at least G^1.

Hence, if we can approximate an input mesh with a G^1, piecewise-quadratic surface, then its contours are piecewise rational functions for orthographic projections.

Next, we summarize the key features of our method, determined by this observation.

Handling Perspective. Many applications require perspective projection, which we handle by applying a projective transformation to the input mesh. That is, an input vertex with camera coordinates $[x,y,z]$ becomes $[x/z, y/z, -1/z]$, with orthographic view vector $\tau = [0,0,1]$, yielding a view equivalent to perspective projection of the original input. A key insight is that, for algebraic contours, we must apply the transformation to the mesh *before* constructing a smooth representation, which makes the surface approximation view-dependent, but does not lead to visible artifacts.

Quadratic Surface Construction. After the projective transformation, we seek to approximate the mesh with a surface composed of quadratic patches. In order for the contours to be continuous across patch boundaries, the surface must be G^1 everywhere, except a small number of isolated points. The existing surface choices are quite constrained. A classical solution to this problem for surfaces parameterized over arbitrary triangulations of the plane is the *Powell-Sabin* interpolant, which was in part motivated by a need for continuous isolines for height fields. Applying these to arbitrary meshes requires several additional components: mesh parametrization, a method for dealing with singular vertices of the parametrization, a way to generate high-quality surfaces without extraneous oscillations, and fast update to handle view-dependence. Our solution is based on the overall idea of He et al [2005], but with several important differences. In particular, their method does not produce quadratic patches for one-ring of triangles near singular vertices of the parametrization (cones) and it is interpolating, rather than approximating, the input mesh.

Visibility. To compute visibility of occluding contours, we adapt the Quantitative Invisibility (QI) algorithm [Appel 1967; Bénard and Hertzmann 2019]. QI is normally applied to mesh contours; we show how to apply it to rational occluding contours, which requires computing curve intersections and solving equations for cusps efficiently.

3.1 Method summary

For a given mesh M, our method begins with preprocessing steps. First, we scale M to fit within a unit box. We then compute a global (u, v) parameterization for M. We split each triangle into 12 triangles, each of which will be a quadratic patch in the final surface. Finally, we precompute a factorized matrix that will allow us to efficiently produce patch coefficients for each new viewpoint, minimizing thin-plate surface energy.

At run-time, given a new camera view $\mathbf{c}$, we perform the following steps: (1) we apply a projective transformation to the mesh M, producing transformed mesh vertices; (2) we use the precomputed matrices to produce Bézier coefficients defining our piecewise quadratic Powell-Sabin surface from the updated vertex position; (3) we then find all contours contained within all patches; (4) we compute their intersections and cusp points, split them at these points and (5) compute visibility using QI. The output rational image space curves may be then rendered in 2D.

Our algorithm allows to perform these steps at a relatively low cost: specifically, (2) requires a single backsubstitution solve for a factorized matrix (3) requires solving a quadratic equation and small linear transformations for patches that may contain contours (4) requires solving a system of two quadratic equations for cusps for each contour segment, and intersecting fourth-degree rational curves for a small number of curve pairs, and (5) tracing a small number of rays intersecting them with quadratic patches (also a system of two quadratic equations).

4 EXTRACTING VISIBLE CONTOURS

Given a C^1 (excluding isolated points) piecewise quadratic surface viewed under orthographic projection, we now show exact procedures for extracting the occluding contour generator and determining which portions are visible. We do assume a general-position view direction, assuming it is perturbed to avoid exact alignment with tangents to flat parts of the surface, and that the surface does not self-intersect. We do *not* assume that the surface itself is in a fully general position: we *do* require that the surface normal does not vanish, except at cones.

4.1 The contours of a single quadratic patch

We first enumerate parametric expressions for the contours of a quadratic surface, ignoring the triangular domain bounds.

For a quadratic surface parameterized by coordinates $\mathbf{r} = [u, v]$, the normal $\mathbf{n}(\mathbf{r})$ is also quadratic. Hence, the contour equation (1) can be written as

$$\tau \cdot \mathbf{n}(\mathbf{r}) = \frac{1}{2}\mathbf{r}^T A\mathbf{r} + \mathbf{b}^T\mathbf{r} + c = 0, \tag{2}$$

i.e., the contours are conic sections in the parametric domain (Figure 3). For completeness, we now enumerate all stable cases, i.e., the ones that are not always eliminated by a perturbation of the view direction.

We diagonalize the contour equation using $A = U^T \mathrm{diag}(\sigma_1, \sigma_2)U$. If A is not singular, then we can reparameterize in terms of unknown $\mathbf{z} = U(\mathbf{r} - A^{-1}\mathbf{b}) = [z_1, z_2]$. The contour equation is then:

$$\sigma_1 z_1^2 + \sigma_2 z_2^2 + \hat{c} = 0. \tag{3}$$

where $\hat{c} = c - \mathbf{b}^T A^{-1}\mathbf{b}/2$. We ensure $\hat{c} \geq 0$ by negating $\sigma_1, \sigma_2, \hat{c}$ if necessary. We then seek a solution curve of the form $\mathbf{z}(t)$. When $\hat{c} > 0$, the solution curve $\mathbf{z}(t)$ may be an ellipse or hyperbola with scales $k_1 = \sqrt{\hat{c}/|\sigma_1|}$, $k_2 = \sqrt{\hat{c}/|\sigma_2|}$. In cone patches (Section 5), $\hat{c} = 0$ occurs stably, in which case the solution $\mathbf{z}(t)$ is a pair of intersecting lines. The first three columns of Table 1 provide the solution curves $\mathbf{z}(t)$ for these cases. Then, the solution curves are converted to parameter domain by $\mathbf{r}(t) = U^T\mathbf{z}(t) + A^{-1}\mathbf{b}$.

If A is singular ($\sigma_2 = 0$), we reparameterize with $\mathbf{z} = U\mathbf{r}$, and the contour equation becomes

$$\sigma_1 z_1^2 + \hat{\mathbf{b}}^T\mathbf{z} + c = 0 \tag{4}$$

where $\hat{\mathbf{b}} = U\mathbf{b}$. We negate terms if $\sigma_1 < 0$. Then, the solution curve $\mathbf{z}(t)$ may be either a parabola or two parallel lines in parameter space, depending on whether $\hat{b}_2 = 0$. Solution curves $\mathbf{z}(t)$ are provided in the fourth and fifth columns of Table 1. The parameter-domain curve is then $\mathbf{r}(t) = U^T\mathbf{z}(t)$. We note that A being near-singular can be stable with respect to the viewpoint change for a cylindrical surface; the contours along a cylinder are always straight lines; see supplemental material for details).

All tests of a quantity x equal to zero are implemented by checking $|x| < \epsilon$, where we use $\epsilon = 10^{-10}$.

Trimming. In general, only a subset of the occluding contour for a quadratic may lie within a patch. We compute the bounds $t \in [t_0, t_1]$ as follows. The above procedure yields a small number of parametric curves of the form $\mathbf{r}(t)$ that we intersect with each of the lines bounding the domain triangle ($u = 0, v = 0, u + v = 1$). If there are intersections, then the curve is broken into any subintervals $[t_{\min}, t_{\max}]$ contained within the triangle, up to three per curve.

Image-space curve. The above steps identify the occluding contour generators within a patch. Each of these are rational curves in 3D, given by $\mathbf{p}(\mathbf{r}(t))$, within bounds $t \in [t_0, t_1]$. Under orthographic projection with the appropriate rotation, projection to image-space curves amounts to removing the third coordinate from a curve, yielding the final result of the computation, a quartic rational curve.

4.2 Visibility of contours on a p.w. quadratic C^1 surface.

To compute curve visibility, we adapt the Quantitative Invisibility (QI) algorithm [Appel 1967]. The QI of a surface point is the number of occluders of that point; a point is visible if and only if it has a QI of zero. Visibility along a curve can only change at cusps, image-space intersections, and contour-boundary intersections. Hence, if we split a curve into segments at each of these cases, then QI for an entire segment can be computed by a single ray-test for the curve. Moreover, the number of ray-tests is minimized by propagating QI through these cases, e.g., using the fact that the QI increases by

Table 1: The occluding contours of a quadratic patch under orthographic projection can be parameterized in one of these five forms, depending on the parameters in Equation 2. Other cases not listed (e.g., σ_1, σ_2>0) do not produce stable contours. See Section 4.1 for details.

A nonsingular ($\sigma_1, \sigma_2 \neq 0$)			A singular ($\sigma_1 \neq 0, \sigma_2 = 0$)	
$\hat{c} > 0, \sigma_1, \sigma_2 < 0$	$\hat{c} > 0, \sigma_1\sigma_2 < 0$	$\hat{c} = 0, \sigma_1\sigma_2 < 0$	$\hat{\mathbf{b}}_2 \neq 0$	$\hat{\mathbf{b}}_2 = 0, \hat{\mathbf{b}}_1^2 > 4\sigma_1^2 c$
Ellipse: $\mathbf{z}(t) = \left[\frac{k_1(1-t^2)}{1+t^2}, \frac{2k_2 t}{1+t^2}\right]$	Hyperbola: $\mathbf{z}(t) = \left[\frac{k_1(1+t^2)}{t^2-1}, \frac{2k_2 t}{t^2-1}\right]$	Intersecting lines: $\mathbf{z}(t) = \left[t, \pm t\sqrt{-\frac{\sigma_1}{\sigma_2}}\right]$	Parabola: $\mathbf{z}(t) = \left[t, \frac{-\sigma_1 t^2 - \hat{b}_1 t - c}{\hat{b}_2}\right]$	Parallel lines: $\mathbf{z}(t) = \left[\frac{-\hat{\mathbf{b}}_1 \pm \sqrt{\hat{\mathbf{b}}_1^2 - 4\sigma_1^2 c}}{2\sigma_1^2}, t\right]$

one from the near side to the far side of a cusp. See [Bénard and Hertzmann 2019] for a full description of the QI algorithm.

Previous work applied QI to triangle mesh contours. In order to compute QI for piecewise-quadratic surfaces, we need new algorithms for computing ray tests, detecting cusps, detecting image-space intersections, and propagating QI.

View graph construction. We first connect contour generator curves across patch boundaries. Since the surface is C^1 by design, the occluding contours will be continuous at patch boundaries, and form disjoint loops, with the exception of cone points, where multiple loops can share a point. We disallow visibility propagation through cone points.

After we compute cusps and image-space contour intersections as explained below, we split the countour curves at these points, constructing a graph of non-intersecting rational quadratic segments with nodes labeled as cusp, intersection, contour-boundary intersection or interior.

Detecting interior cusps. To determine cusp positions in the *interior* of a quadratic patch, we need to find points where τ is parallel to the tangent to the contour curve. One can use this definition directly, solving the univariate equation $\tau \cdot \frac{d}{dt}\mathbf{p}(\mathbf{r}(t)) = 0$; however, this equation is a degree 10 polynomial. Instead we opt for a different approach. We introduce two orthogonal vectors τ_1 and τ_2 perpendicular to the view direction, and define the cusp as the point on the contour for which the tangent is orthogonal to both of these vectors. An important observation is that the unnormalized tangent to all isolines of $\mathbf{n} \cdot \tau$ in the parametric domain is given by $[-\mathbf{n}_v \cdot \tau, \mathbf{n}_u \cdot \tau]$, which is a linear function on the whole patch. Mapped to 3D this yields a quadratic equation:

$$\mathbf{t}(u, v) = -\mathbf{p}_u(\mathbf{n}_v \cdot \tau) + \mathbf{p}_v(\mathbf{n}_u \cdot \tau) \tag{5}$$

i.e., the components of the tangent are also quadratic, and the condition for a cusp is a system of two quadratic equations in (u, v):

$$\mathbf{t}(u, v) \cdot \tau_1 = 0, \text{ and } \mathbf{t}(u, v) \cdot \tau_2 = 0, \tag{6}$$

Note that the contour equation itself is redundant, because if the view direction is aligned with a surface tangent, it is perpendicular to the normal. We solve this equation using the pencil method which reduces it to a cubic equation in one variable, and a pair of quadratic equations, which we also use for ray-patch intersections [Ogaki and Tokuyoshi 2011]; the cubic equation is solved by finding the companion matrix eigenvalues with QR decomposition.

QI increases by 1 along the contour in the direction of $\mathbf{t}$.

Figure 4: Two types of cusps, a cusp interior to a patch, and a boundary cusp.

Patch-boundary cusps. At a patch boundary, a contour may have two distinct tangents $\mathbf{t}$ and $\mathbf{t}'$ corresponding to the common endpoint of the contour segments in two patches; a cusp occurs if $\tau \cdot (\mathbf{n} \times \mathbf{t})$ and $\tau \cdot (\mathbf{n} \times \mathbf{t}')$ have different signs. While in this case there is no need to split either of the curves at the cusp, the visibility may change at the common endpoint, which we refer to as a *boundary cusp*. We opt not to propagate visibility through such cusps.

Image-space contour intersections. The intersections of contours in the image domain are computed using the standard Bézier clipping algorithm [Sederberg and Nishita 1990], which typically converges in a few iterations. The QI on the top contour is a constant q, the segment of the lower contour not occluded by the near surface also has QI q, and the lower contour segment occluded by the surface has QI $q + 2$.

Ray tests and QI propagation. Once the contours have been split at cusps and intersections, the visibility can be determined by performing a ray test at a contour segment and then propagating QI. We do not propagate QI through cone points, since multiple contours may meet there, and we do not propagate QI though patch-boundary cusps. If any segments do not have QI values assigned after propagation, the process repeats with a new ray test. We use a highly efficient ray-quadratic patch intersection test [Ogaki and Tokuyoshi 2011]; however, a substantial speed up is obtained using the standard bounding-box tests and spatial grid for acceleration.

5 SURFACE CONSTRUCTION

We now describe our smooth surface construction; producing a high-quality surface is essential to generating well-behaved contours. The input is an arbitrary manifold mesh $M = (V, E, F)$ possibly with boundary, along with vertex positions. The output is a surface composed of quadratic patches joined with G^1 continuity,

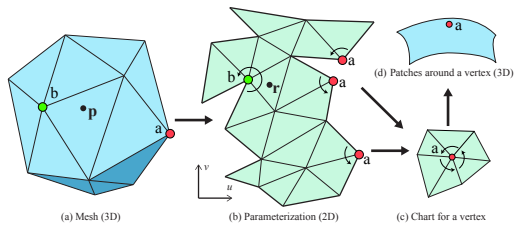

Figure 5: Surface definition. (a) An input mesh with vertex positions $\mathbf{p}_i$. (b) 3D mesh is mapped to the plane with cuts with texture coordinates $\mathbf{r} = (u, v)$, consistent edge length and sum of angles at most vertices equal to 2π (c) Vertex a (red vertex) occurs at three places on the cut, but a chart can be constructed where the sum of the angles around a is 2π. For a cone vertex b (green vertex), the sum is less than 2π. (d) The chart for each vertex with respect to which gradient DOF are defined.

everywhere except at a small number of isolated vertices. Our approach is based on He et al. [2005], with several important changes.

5.1 Conformal parameterization

We first compute a locally bijective global surface parameterization, to produce uv coordinates for the mesh, as required for the Powell-Sabin interpolant. More specifically, the parameterization is a flattening into the uv plane of the input mesh M (Figure 5). The mesh is cut to a disk M^c along mesh edges, such that, for two images of a cut edge in the uv domain, their lengths match. The metric is almost everywhere flat if, for all vertices—including all cut vertices, but excluding a small number of *cone vertices*—the sum of the angles in the parametric domain is 2π.

The first step is to select a small number of cone vertices, consistent with genus; we use $4(g-1)$ cones for $g > 1$ and 8 for genus zero. We uniformly distribute them over the surface, by partitioning it into clusters of close size, and, for each cluster, picking the vertex of smallest discrete curvature, as this minimizes visual artifacts of the surface. We do not fix angles at the cones; instead, we fix the conformal scale factors at cones to 1, as in [Springborn et al. 2008], to minimize parametrization distortion at these points. Additionally, we perform one step of Loop subdivision on the incident triangles, to further reduce artifacts.

Then, we apply an efficient implementation of discrete conformal maps [Campen et al. 2021; Gillespie et al. 2021] mathematically guaranteed [Gu et al. 2018] to satisfy these requirements, possibly with a moderate amount of mesh refinement. These methods compute an edge length assignment, which we convert to a uv parametrization using a greedy layout algorithm, as in [Springborn et al. 2008].

While conformal parameterization can have extreme scale variation, uniformly scaling the parametric domain of a polynomial patch does not affect its shape, and so this scaling does not significantly impact our output surface construction. This stage of our algorithm differs from the parameterization step of He et al. [2005], which does not guarantee injectivity, fixes cone angles to multiples of π, and does not provide control over their position, leading to higher distortion.

5.2 Powell-Sabin construction

The Powell-Sabin interpolant transforms a set of input degrees-of-freedom (DOFs) q associated with a mesh M into a set of quadratic patches $\mathbf{p}(u, v)$. Each patch is defined by six Bézier points on a triangle in the uv plane. Each of these triangles is obtained by splitting the parametric images of the triangles in the original mesh (Figure 6(right)). We use the more-expensive 12-split Powell-Sabin construction as it yields higher surface quality, but a 6-split can also be used with acceptable results. All quadratic patches associated with an input triangle form a C^1 piecewise-quadratic macropatch $\mathbf{P}(u, v)$.

The input coefficients include, for every vertex, the position $\mathbf{p}$ and two tangents $\mathbf{g}^u, \mathbf{g}^v$, and for every edge midpoint, a single tangent $\mathbf{g}^m$ (Figure 6(left)). If the parametrization has no cuts, then $\mathbf{g}^u$, $\mathbf{g}^v$ are prescribed values of $\partial_u \mathbf{P}(u_i, v_i)$ and $\partial_v \mathbf{P}(u_i, v_i)$ at a vertex i respectively, and $\mathbf{g}^m$ is $\partial_{\mathbf{e}_{ij}^\perp} \mathbf{P}(u, v)$, where $\mathbf{e}_{ij}^\perp$ is the vector perpendicular to the edge $\mathbf{e}_{ij}$. All macropatches $\mathbf{P}$ sharing a vertex share these coefficients at the vertex, and similarly, when two patches sharing a midpoint, they share a coordinate derivative in the direction $\mathbf{e}_{ij}^\perp$. Powell and Sabin [1977] show that sharing these degrees of freedom is sufficient for their construction to yield a C^1 surface.

We write the complete set of free DOFs for the surface as a vector of coefficients:

$$q = [\mathbf{p}_1, \mathbf{g}_1^u, \mathbf{g}_1^v \cdots \mathbf{p}_n, \mathbf{g}_n^u, \mathbf{g}_n^v, \mathbf{g}_1^m \cdots \mathbf{g}_m^m] \tag{7}$$

where n is the number of vertices, m the number of edges, and $\mathbf{g}_\ell^m = \mathbf{g}_{ij}^m$ if the edge (ij) has index ℓ; the 3 components of each vector are flattened.

Local parameters. To construct a single macropatch for a triangle (ijk) we extract the relevant 12 DOFs from this vector to obtain a *local* DOF vector, $q^{\text{loc}} = [\mathbf{p}_i, \mathbf{g}_i^u, \ldots \mathbf{g}_k^m]$, with 12 vector DOFs; the coefficients of the 12 quadratic subpatches are obtained by the standard Powell-Sabin linear transformation of this vector, described in supplementary: this yields six Bezier points for each quadratic patch as shown in Figure 6.

$$p^\ell = B_\ell q^{\text{loc}}, \ \ell = 1 \ldots 12$$

where each B_ℓ is a 6×12 matrix applied coordinatewise, mapping twelve local degrees of freedom to six quadratic patch coefficients.

Cut vertices. To determine local DOFs q^{loc} from global q at a vertex i or edge (ij) on the cut, we need to define the local gradients $\mathbf{g}$ consistently across the cut. We do so by constructing a chart comprising all incident triangles of the vertex or edge. One of the edges of the chart is arbitrarily chosen to be the u' coordinate direction, defining a local coordinate system (u', v'). All triangles are mapped to the chart by a rigid transformation T_{ijk} from global $\mathbf{r} = (u, v)$ to chart coordinates $\mathbf{r}' = (u', v')$ (Figure 5(c)). Constructing such a common domain is possible because the parameterization (Section 5.1) produces angles that sum up to 2π and edge lengths that match across the cut.

Then we can obtain the gradient DOFs $\mathbf{g}$ in coordinates (u', v') on the chart. To obtain the vertex DOFs for each triangle (ijk), we

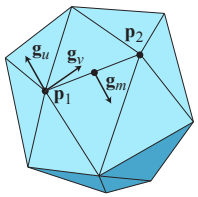

Figure 6: Left: The smooth surface is parameterized by the following coefficients: for each vertex, a position p and gradients $\mathbf{g}_u$ and $\mathbf{g}_v$, and, for each edge, a midpoint gradient $\mathbf{g}_m$. Right: Each input triangle is split into 12 triangles, with Bézier controls named as shown here. These control points are computed from the smooth surface coefficients $q^{\text{loc}} = [\mathbf{p}_i, \mathbf{g}_i^u, ..., \mathbf{g}_k^m]$.

Figure 7: Surface and contour behavior near a cone. (Open source Helmet model from OpenSubdiv.)

apply T_{ijk} and apply the corresponding transform to $\nabla_{\mathbf{r}'}\mathbf{P}(\mathbf{r}') = [\partial_{u'}\mathbf{P}, \partial_{v'}\mathbf{P}]$, to obtain the local DOF vector $\nabla_{\mathbf{r}}\mathbf{P} = T_{ijk}\nabla_{\mathbf{r}'}\mathbf{P}$ for the macropatch. This construction retains C^1 continuity across cut edges and at non-cone vertices [He et al. 2005].

Cone vertices. At cone vertices, we create degenerate patches, e.g., [Neamtu and Pfluger 1994]. In order to achieve consistent gradients across the cut, we set gradients $\mathbf{g}_u, \mathbf{g}_v$ to zero at cone vertices, making the patch map $\mathbf{p}(u, v)$ singular at these points. The conditions for C^1 continuity are still satisfied, although the surface has a cone point at the vertex (Supplemental Material). While one could apply spline surface construction methods that force a common tangent plane at a singular vertex, we found our approach to have negligible impact on contour quality. Typical surface behavior near a cone is shown in Figure 7.

5.3 Thin-plate optimization

We now describe how to optimize the global parameters q to produce a smooth surface shape that approximates the input mesh vertices. One nice property of a Powell-Sabin spline surface is that it is C^1, and, at the same time, minimal degree. C^1 is a requirement for *conforming* finite-element discretization of optimization objectives containing second derivatives, in particular, the thin-plate functional:

$$E(q) = \sum_{\ell} \int_{\Omega_\ell} \mathbf{p}_{uu}^2 + \mathbf{p}_{vv}^2 + 2\mathbf{p}_{uv}^2 dudv + w \sum_i A_i(\mathbf{p}_i - \mathbf{p}_i^0)^2 \quad (8)$$

where $\mathbf{p}_i^0$ are the input mesh vertices and Ω_ℓ is the ℓ-th triangle domain. We discretize the functional by simply substituting the expressions for quadratic patches in terms of the degrees of freedom q^{loc}, yielding constant expressions for the integrands for each quadratic patch. These expressions are quadratic in the degrees of freedom, so the energy can be written as a quadratic form for each patch with a constant local matrix, which are then assembled into a global system. We pre-compute matrices H^s and H^f, combined into a matrix $H = H^s + wH^f$, corresponding to the fitting and smoothing terms of the quadratic objective (8) respectively. Then the energy can be written in the form

$$E(q) = \frac{1}{2}(q^T H^s q + w(q - q_0)^T H^f (q - q_0)) \quad (9)$$

$$= \frac{1}{2} q^T H q - w q^T H^f q_0 + \text{const} \quad (10)$$

The constant term q_0 is the vector of degrees of freedom with the original mesh vertex positions for $\mathbf{p}_i$ and zeros for all other components. This objective is minimized by solving $Hq = wH^f q_0$ for q, i.e. a single sparse linear solve.

One can also show by direct computation that our optimization objective minimum, expressed as Bézier control points of patches, is independent of the choice of the chart coordinates (u', v').

Efficient computation. Changes to viewpoint change the mesh vertex positions due to the projective transformation (Section 3), thus changing q_0. However, the matrix H depends on parametric coordinates and *not* on the vertex positions. Hence, all of the computations in this section, including mesh parameterization and defining local and global coordinates, can be computed in a preprocess. We precompute the Cholesky decomposition of the sparse matrix H, which makes the cost of solving the system $Hq = wH^f q_0$ negligible at run-time.

6 EVALUATION

We show results of our method in Figures 1 and 8. Our method produces clean, smooth rational output curves without gaps or other topological errors.

Timings and scaling. We tested our algorithm on the same small test set that was used in [Liu et al. 2023]. We preprocessed the meshes by performing boolean unions of parts to eliminate all intersections, and cleaning up the resulting mesh to eliminate very short edges. Figure 11 shows the timings for the view-dependent and view-independent parts of our algorithm. We use 29 models and 26 randomized views per model. We see relatively very little per-frame timing variation (standard deviation less than 150 ms for the largest models), and both precomputation and per-view performance scales approximately linearly with the input size. Our implementation is serial and not heavily optimized, and we expect that performance can be significantly improved.

Comparison with previous work. Precise comparison with ConTesse [Liu et al. 2023], the state-of-the-art method for accurate contours, is difficult for various reasons, e.g., that paper reports timing only for mesh generation. Nonetheless, it is clear that our approach operates an order of magnitude faster during run-time, and scales much better as well. Our entire per-view processing

time, for smaller meshes, is comparable to ConTesse's mesh generation step alone. However, for larger meshes, our method becomes orders-of-magnitude faster, e.g., for "Fertility," ConTesse averages 16 seconds per view for mesh generation, whereas we require 0.4 seconds per view for the entire visibility pipeline, after a 4.4 seconds of preprocessing time. For "Killeroo," ConTesse requires 33 seconds per view for mesh generation, whereas we require 0.5 seconds per view, after 3 seconds preprocessing. We evaluated timing on an MacBook Pro 2.7 GHz, an older machine than the ConTesse results are reported on.

This is to be expected, since our method requires only a linear solve to compute patch coefficients for each new view, whereas ConTesse employs many iterative heuristic search steps to find a valid mesh for each new view.

View-dependent effects. Because we construct a view-dependent surface approximation, in principle, the object could appear non-rigid during rotation. Figure 10 and the accompanying video show that the projective transformation does not alter the geometric appearance or produce non-rigid effects.

Smoothness vs. approximation. All examples in this paper were generated using a constant weight $w = 1$ for the fitting term in (10); Figure 9 shows how the surface changes as this weight varies.

7 CONCLUSIONS

We have presented a method for efficient computation of high-quality occluding contours on C^1 surfaces approximating arbitrary input meshes. As contours are computed in closed algebraic form and are the exact (modulo numerical errors) contours of the surface, visibility computation is a straightforward extension of the QI algorithm for meshes, and, at the same time, are close to the contours on the smooth surface that the mesh is approximating. The resulting contour lines cannot violate topological conditions from [Liu et al. 2023], except due to numerical error for non-general-position choices of the view direction. Hence, the output contours have valid, well-defined visibility, without the artifacts that have plagued previous methods.

There are many extensions that are easy to add to our framework: it is straightforward to add sharp features, and handle non-manifold surfaces, e.g., resulting from intersections or self-intersections. Other curves, like suggestive contours, apparent ridges [DeCarlo 2012], and can be added as polylines, as well as stylized shading effects. The per-view computational cost of our approach is proportional to the number of contour segments and is embarrassingly parallel: an optimized implementation is likely to improve performance by a large factor. Another potentially important direction is to explore higher-order constructions with constraints with reducible parametric contour equations, as these may eliminate the need for a global parametrization.

REFERENCES

Arthur Appel. 1967. The Notion of Quantitative Invisibility and the Machine Rendering of Solids. In *Proceedings of the 1967 22nd National Conference (ACM '67)*. ACM, 387–393. https://doi.org/10.1145/800196.806007

Pierre Bénard and Aaron Hertzmann. 2019. Line Drawings from 3D Models. *Foundations and Trends in Computer Graphics and Vision* 11, 1-2 (2019), 1–159. https://doi.org/10.1561/0600000075

Pierre Bénard, Aaron Hertzmann, and Michael Kass. 2014. Computing Smooth Surface Contours with Accurate Topology. *ACM Trans. Graph.* 33, 2, Article 19 (2014), 21 pages. https://doi.org/10.1145/2558307

Marcel Campen, Ryan Capouellez, Hanxiao Shen, Leyi Zhu, Daniele Panozzo, and Denis Zorin. 2021. Efficient and robust discrete conformal equivalence with boundary. *ACM Transactions on Graphics (TOG)* 40, 6 (2021), 1–16.

Roberto Cipolla and Peter Giblin. 2000. *Visual Motion of Curves and Surfaces*. Cambridge University Press.

Wolfgang Dahmen. 1989. Smooth piecewise quadric surfaces. In *Mathematical methods in computer aided geometric design*. Elsevier, 181–193.

Doug DeCarlo. 2012. Depicting 3D shape using lines. In *Proc. SPIE*, Vol. 8291. 8291 – 8291 – 16. https://doi.org/10.1117/12.916463

Philippe Decaudin. 1996. *Cartoon Looking Rendering of 3D Scenes*. Research Report 2919. INRIA. http://phildec.users.sf.net/Research/RR-2919.php

Elmar Eisemann, Holger Winnemöller, John C. Hart, and David Salesin. 2008. Stylized Vector Art from 3D Models with Region Support. In *Proceedings of the Nineteenth Eurographics Conference on Rendering (EGSR '08)*. Eurographics Association, 1199–1207. https://doi.org/10.1111/j.1467-8659.2008.01258.x

Gershon Elber and Elaine Cohen. 1990. Hidden Curve Removal for Free Form Surfaces. In *Proceedings of the 17th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '90)*. ACM, 95–104. https://doi.org/10.1145/97879.97890

Rafael Sendra Ferrer, Sonia Pérez Díaz, and F Winkler. 2008. *Rational algebraic curves: a computer algebra approach*. Springer Berlin.

Mark Gillespie, Boris Springborn, and Keenan Crane. 2021. Discrete conformal equivalence of polyhedral surfaces. *ACM Transactions on Graphics* 40, 4 (2021).

Xianfeng David Gu, Feng Luo, Jian Sun, and Tianqi Wu. 2018. A discrete uniformization theorem for polyhedral surfaces. *Journal of Differential Geometry* 109, 2 (2018), 223–256.

Ying He, Miao Jin, Xianfeng Gu, and Hong Qin. 2005. A C^1 globally interpolatory spline of arbitrary topology. In *International Workshop on Variational, Geometric, and Level Set Methods in Computer Vision*. Springer, 295–306.

Aaron Hertzmann and Denis Zorin. 2000. Illustrating Smooth Surfaces. In *Proceedings of the 27th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '00)*. ACM Press/Addison-Wesley Publishing Co., 517–526. https://doi.org/10.1145/344779.345074

Tobias Isenberg, Nick Halper, and Thomas Strothotte. 2002. Stylizing silhouettes at interactive rates: From silhouette edges to silhouette strokes. In *Computer Graphics Forum*, Vol. 21. 249–258. https://doi.org/10.1111/1467-8659.00584

B. Jüttler. 1998. Triangular Bézier surface patches with a linear normal vector field. In *The mathematics of surfaces, VIII (R. Cripps, ed.)*. Info. Geom., Winchester, 431–446.

B. Jüttler and M.L. Sampoli. 2000. Hermite interpolation by piecewise polynomial surfaces with rational offsets. *Computer Aided Geometric Design* 17, 4 (2000), 361–385.

D. Kirsanov, P. V. Sander, and S. J. Gortler. 2003. Simple Silhouettes for Complex Surfaces. In *Proceedings of the 2003 Eurographics/ACM SIGGRAPH Symposium on Geometry Processing (SGP '03)*. Eurographics Association, 102–106.

Chenxi Liu, Pierre Bénard, Aaron Hertzmann, and Shayan Hoshyari. 2023. ConTesse: Accurate Occluding Contours for Subdivision Surfaces. *ACM Trans. Graph.* 42, 1, Article 5 (Feb. 2023), 16 pages. https://doi.org/10.1145/3544778

Marian Neamtu and Pia R Pfluger. 1994. Degenerate polynomial patches of degree 4 and 5 used for geometrically smooth interpolation in 3. *Computer Aided Geometric Design* 11, 4 (1994), 451–474.

J. D. Northrup and Lee Markosian. 2000. Artistic Silhouettes: A Hybrid Approach. In *Proceedings of the 1st International Symposium on Non-photorealistic Animation and Rendering (NPAR '00)*. ACM, 31–37. https://doi.org/10.1145/340916.340920

Shinji Ogaki and Yusuke Tokuyoshi. 2011. Direct Ray Tracing of Phong Tessellation. *Computer Graphics Forum* 30, 4 (2011), 1337–1344. https://doi.org/10.1111/j.1467-8659.2011.01993.x

Michael JD Powell and Malcolm A Sabin. 1977. Piecewise quadratic approximations on triangles. *ACM Transactions on Mathematical Software (TOMS)* 3, 4 (1977), 316–325.

Takafumi Saito and Tokiichiro Takahashi. 1990. Comprehensible Rendering of 3-D Shapes. In *Proceedings of the 17th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '90)*. ACM, 197–206. https://doi.org/10.1145/97879.97901

Thomas W Sederberg and Tomoyuki Nishita. 1990. Curve intersection using Bézier clipping. *Computer-Aided Design* 22, 9 (1990), 538–549.

Boris Springborn, Peter Schröder, and Ulrich Pinkall. 2008. Conformal equivalence of triangle meshes. *ACM Transactions on Graphics (TOG)* 27, 3 (2008), 1–11.

Matei Stroila, Elmar Eisemann, and John Hart. 2008. Clip Art Rendering of Smooth Isosurfaces. *IEEE Transactions on Visualization and Computer Graphics* 14, 1 (2008), 135–145. https://doi.org/10.1109/TVCG.2007.1058

Ruth A. Weiss. 1966. BE VISION, A Package of IBM 7090 FORTRAN Programs to Draw Orthographic Views of Combinations of Plane and Quadric Surfaces. *J. ACM* 13, 2 (1966), 194–204. https://doi.org/10.1145/321328.321330

Georges Winkenbach and David H. Salesin. 1996. Rendering Parametric Surfaces in Pen and Ink. In *Proceedings of the 23rd Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '96)*. ACM, 469–476. https://doi.org/10.1145/237170.237287

Figure 8: Examples of of contour line sets obtained using our method on models from the modified dataset of [Liu et al. 2023]. (Bigguy and Monster Frog ©Bay Raitt. Fertility courtesy UU from AIM@SHAPE-VISIONAIR Shape Repository. Public domain Blub, Bob, and Ogre models by Keenan Crane. Killeroo ©headus.com.au. Open source Pawn model from OpenSubdiv.)

Figure 9: Given a triangle mesh, we need a representation of the underlying smooth surface. Here we visualize the dependence of our smooth surface shape fitting on the weight w, without a projective transformation step. We use $w = 1$ for all experiments in this paper. (Public domain Ogre model by Keenan Crane.)

Figure 10: Perspective projection of a mesh, vs. orthographic projection of the projective-transformed mesh. In each pair, the two renderings are geometrically equivalent. (The shading differs, but is not used in contour detection and shown here only for visualization. Killeroo ©headus.com.au.)

Figure 11: Timing for our method, on meshes from the ConTesse [Liu et al. 2023] dataset. Orange circles show preprocessing computation time, and blue circles show per-frame time, averaged over 26 frames. While preprocessing can take several seconds, per-frame computation is very efficient, growing very slowly as a function of mesh size. Computation times were measured on a MacBook Pro 2.7Ghz Intel Core i7, 16Gb memory. The "car" mesh is more intensive at run-time due a very high shape complexity, requiring over a million ray-patch intersection tests per frame.